

containers

classmate

Date _____
Page _____

↳ why do we need containers?

Evolution of Infrastructure.

i] Physical servers.

- Initially, applications were deployed directly on physical servers.

ii] Virtual machines

- To improve resource utilization, organizations hypervisor ~~started us~~ on top of a physical server.

Example:-

Example:- A physical server has 100 GB RAM and 100 CPU units (example units)

- Using a hypervisor, 4 VMs (VM1, VM2, VM3, VM4) are created

- Each VM is allocated 25 GB RAM.

- But, later, organizations realized that many virtual machines were not utilizing their full allocated resources.

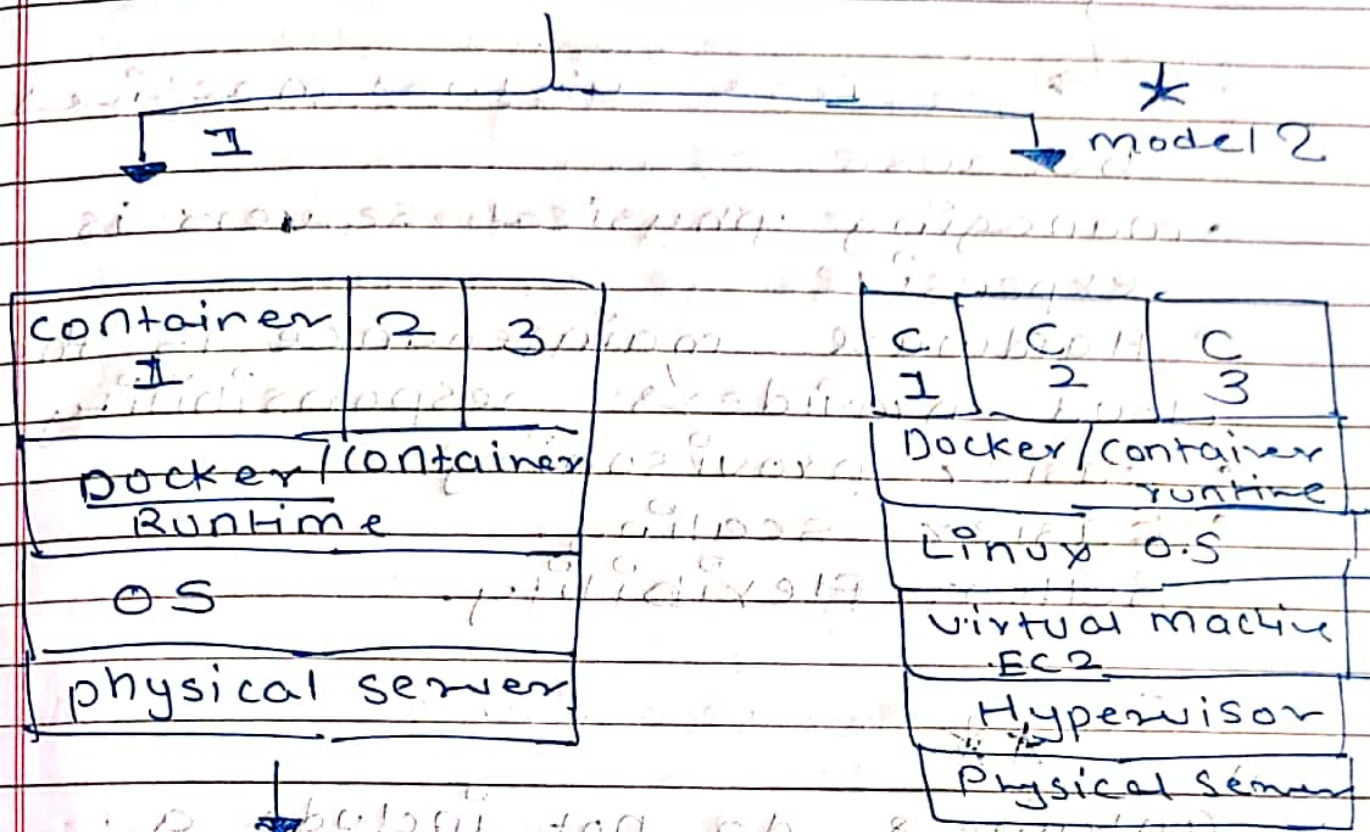
- In DevOps environment, there are 100s or 1000s of such underutilized virtual machines.

③ Containers :- To address the resource utilization of VM, containers were introduced. Virtual machines solved many problems of physical servers.

- Containers solved many problems of VM, but only to an extent. Why extent?

- Containers has security drawback. Each VM has own OS, providing strong isolation.
- Containers have logical isolation, but not strong as VMs.
- Containers share host OS, which means they can share certain resources with the host.

Types of Model



eg:-

IBM server
or
HP server

↓
Linux operating system

↓
Docker

↓
containers

eg:- Physical server

↓
Hypervisor

↓
EC2 instance

↓
Linux

↓
Docker

↓
containers

organizations prefer model 2

↳ prefers virtual machines because

- managing physical servers is expensive
- Hardware maintenance is the cloud providers responsibility.
- Easier provisioning
- Easier scaling
- Better flexibility.

★★

Containers do not include a complete operating system

↓
they include a minimal user-space file system (base image) while sharing the host kernel.

Base Image

↓
It's a package containing

- Application
- Application Libraries
- Application dependencies
- System Libraries
- minimal userspace files.

It serves as the starting point for creating a container

• Why are Containers Easy to Move?

virtual machine backup



Snapshot

↳ Usually ~~2GB~~ → 2GB

container

image

• 3GB

• or even bigger



usually

• 100 MB

• 300 MB

Containers are much smaller,

they are

• easy to copy

• easy to transfer

• easy to deploy

This process is called shipping

moving container image ← means
from one environment to another

Example → Developer Laptop → Docker Registry

Kubernetes
Cluster

Docker
Host

— why is Docker Popular?

provides:

- simple interface
- Easy commands
- Easy image creation
- Easy container management
- Large Ecosystem.

Docker workflow

↓
Lifecycle is as follows

write Dockerfile

↓
Docker Engine builds Image

↓
Docker Image

↓
Docker Engine runs Image

↓
Docker container

Simple workflow

Dockerfile → docker build

→ Docker Image

Run

Docker container

Docker Engine

↳ It is responsible for:-

- Reading Docker files
- Building Images
- Running Containers
- Managing Containers

Diagram



Docker commands



docker build

docker run

docker stop

docker ps



Docker
Engine



→ Build Image

→ Run Container

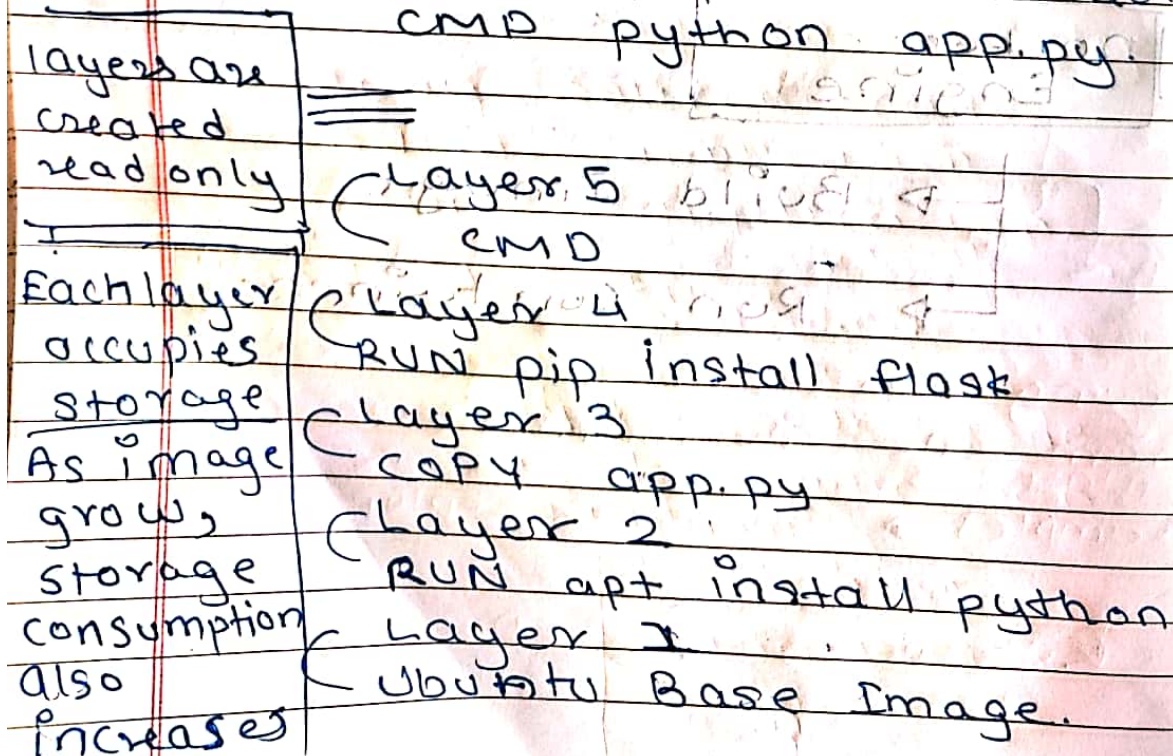
Docker Image Layers

↳ When Docker builds an image, it creates multiple read-only layers.

↓
Each Dockerfile instruction typically creates a new layer.

Example :- Dockerfile

```
FROM UBUNTU
RUN apt install python
COPY app.py
RUN pip install flask
CMD python app.py
```



Buildah

↳ It's an alternative image-building tool.

↳ Its goals include:

- Building Docker-compatible images
- OCI-compliant images
- Scriptable image creation
- Better integration with Podman
- * • Daemonless image building

- Instead of writing Dockerfiles only
- Buildah also allows image creation using shell script and Buildah commands.

↳ It provides more flexible and daemonless ways to build & manage image layers.

multi-stage Builds

& Distroless Images

Imagine you have developed a Python calculator Application.

& you have goal simply run this application inside a Docker container.

Beginner Dockerfile would something like this:

From Ubuntu

Install Python

— pip

— all dependencies

Copy source code

Build application

Run application

High level workflow

Python Source Code



Ubuntu Image

- Install Python
- pip
- dependencies
- Copy code
- Build app
- Run app



this works perfectly **BUT**

There's a problem.

our application only needs

- Python Runtime
- Application Code

But our Docker image also contains

- Ubuntu OS
- apt package manager
- Build tools
- Temporary Files
- Cache
- Extra Linux libraries

which are not required once the application has been built.

This is the biggest reason Docker introduced multi-stage Builds

— Build Stage vs Runtime Stage

Example :- During construction you need

- Cement mixer
- cranes
- workers
- Scaffolding.

After the house is complete do you keep all those inside your living room?

No → we remove everything except the final house

Docker follows exactly the same concept

— Build stage is responsible for:-

- Installing dependencies.
- Installing compilers
- Building binaries
- Packaging application.

Runtime stage :-

- ↳ Responsible for only one thing.
- ↳ Run Application.

Multi-stage Docker build

- ↳ contains multiple FROM statements.

Example :-

FROM ubuntu

Install everything
Build application.

} STAGE 1

STAGE 2

FROM python-runtime
COPY binary from Stage 1
RUN application

This is still one Docker file, but
it contains multiple stage:

• with multi-stage build:

small image { Python Runtime
+ Application

Everything used only during building stays behind.

Only what is needed for execution reaches production.

Another Example :-

(THREE-TIER Application)

An application consisting of

- REACT FRONTEND
- Spring Boot Backend
- MySQL

For this, we need to install:

• Ubuntu

• Java

• NodeJS

• React

• MySQL

• npm

• Maven

• Build tools

Everything stays inside image.

Final image size = 1-1.5 GB

In multi-stage version

stage 1

Install Java
Install Maven
Install Node
Install Frontend
Install Backend
Install app.eax

STAGE 2

FROM openjdk Runtime
COPY app.eax
RUN app.eax

Final Image :- 150-200 MB

multiple stages

Many people think Docker only supports two stages:- That's wrong

→ Docker file may contain:-

Stage 1

Stage 2

Stage 3

Stage N

Final stage

Example:

Frontend Build



Backend Build



Testing



Security Scan



Packaging



Production Runtime

— Only image becomes the final Docker Image is the last stage.

There is only one final image produced by Docker build & it is typically created from the last stage. In production, this final stage is usually kept as minimal as possible to reduce image size and improve security.

Distroless Image.

classmate

Date _____

Page _____

— A Distroless image contains only what is required to execute the application.

— Example — Python Distroless

contains

↳ Python Runtime.

Does not contain:

- apt
- bash
- shell
- package manager
- debugging utilities.

→ Distroless improves security.

- Even if a hacker grants access to container they usually try commands like `ls`, `find`, `curl`, `bash`, `sh`.

• Most of these utilities do not exist inside a distroless image.

- Attack surface is reduced.
- unnecessary software is removed.
- Fewer vulnerabilities exist.

Docker : Volumes & Bind mounts

Containers are short-lived
They can be created, stopped
or removed any-time.

Problem 1 — An NGINX container stores authentication logs such as:

- User login details
- IP addresses
- Access logs

These logs are required for:

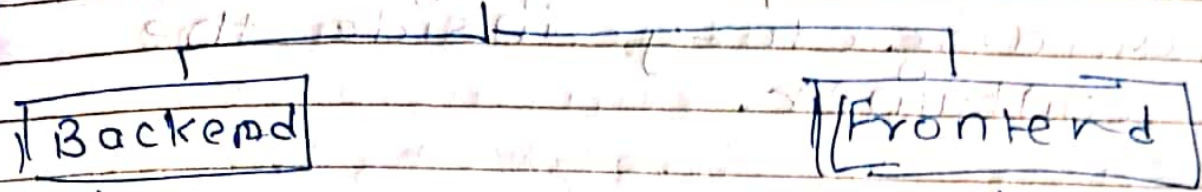
- Auditing
- Troubleshooting
- Performance Analysis
- compliance (eg. keeping logs for 30 days)

So, if container is removed
Results in:-

- Audit history is lost
- Previous user activity cannot be recovered.

② Another Real-world problem.

suppose, we have 2 containers



creates file

Reads files.

the backend continuously generates HTML reports.

Example :- reports/

today.html

yesterday.html

lastweek.html

• The frontend serves these files.

≡ If container gets removed

• User cannot access older reports

• The application behaves incorrectly because historical data has disappeared.

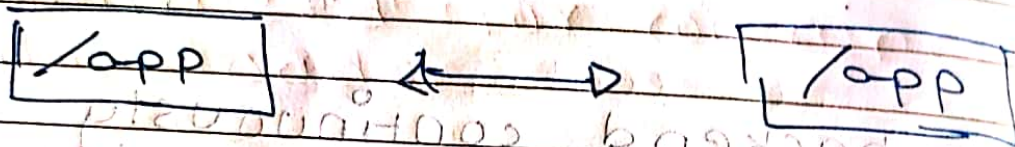
SOLUTION

Docker Bind mounts

→ A Bind mount connects a directory on the host to a directory inside the container.

Host
machine

Container



The folder name can be same or different.

→ Whenever the container writes files inside its mounted directory, the data is actually stored on the host.

Even if the container is removed, the data remains safe.

When a new container starts, it can mount the same host directory and continue using the existing data.

Key points :-

- maps a host directory to a container directory.
- Host path is specifically manually.
- data survives container removal
- commonly used during development.

Docker volumes.

- ↳ are Docker-managed persist + storage
- ↳ It is not a disk partition.

It is a storage location (directory) on the host, managed independently of containers.

Docker → creates volume.

mounted into container. ← volume

Advantages:

- managed entirely using Docker CLI
- can be attached to one or multiple containers
- Easy to inspect, backup and remove
- can integrate with external storage drivers
- optimized for container workloads.

Volume Commands

List — docker volume ls

Create — docker volume create

Inspect a volume — docker volume inspect

Created at;

Driver

Labels

Mountpoint

Name

Options

Scope

tells you where Docker stores the volume on the host.

Remove a volume — docker volume rm Pratik

Bind mount

- Host path is manually specified

- depends on the host directory

- Good for development

- Data remains after container removal

- Easier to access host files directly

Docker volume

Docker manages the storage location

Managed completely by Docker

Recommended for production persistent data

Data remains after container removal

Easier to manage with Docker CLI

Docker Networking

classmate
Date _____
Page _____

→ Imagine Docker is installed on your server (physical EC2 etc)

→ Suppose you have 2 containers

C1 → (Frontend / Logic Application)
C2 → (Backend API)

The frontend needs to communicate with backend without networking, these containers would have no way to communicate with each other.

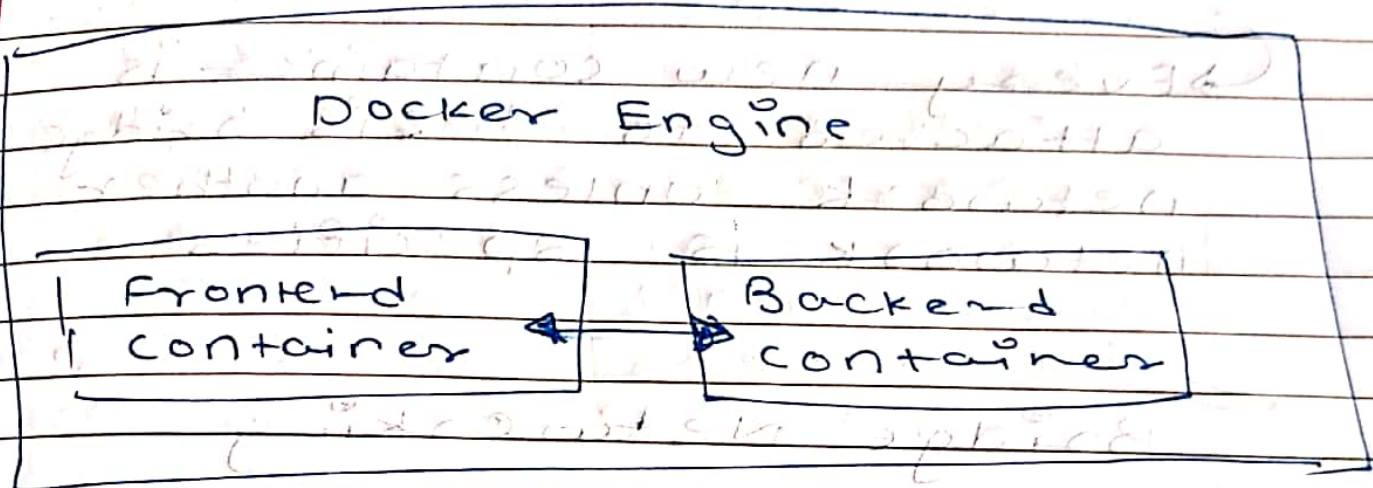
Docker networking provides this communication mechanism



Container ↔ Container
Container ↔ Host communication

Basic Architecture

Host machine (EC2, VM, Physical)



without Docker networking, the frontend container cannot communicate with the backend container.

So, what happens when a container is created?

↓
When Docker creates a container, it connects that container to network.

↳ So, by default Docker uses Bridge networking.

classmate
Date _____
Page _____

Whenever Docker creates a default bridge network ~~is~~ is created when docker daemon starts.

↳ Every new container is attached to this bridge network unless another network is specified.

Bridge Networking

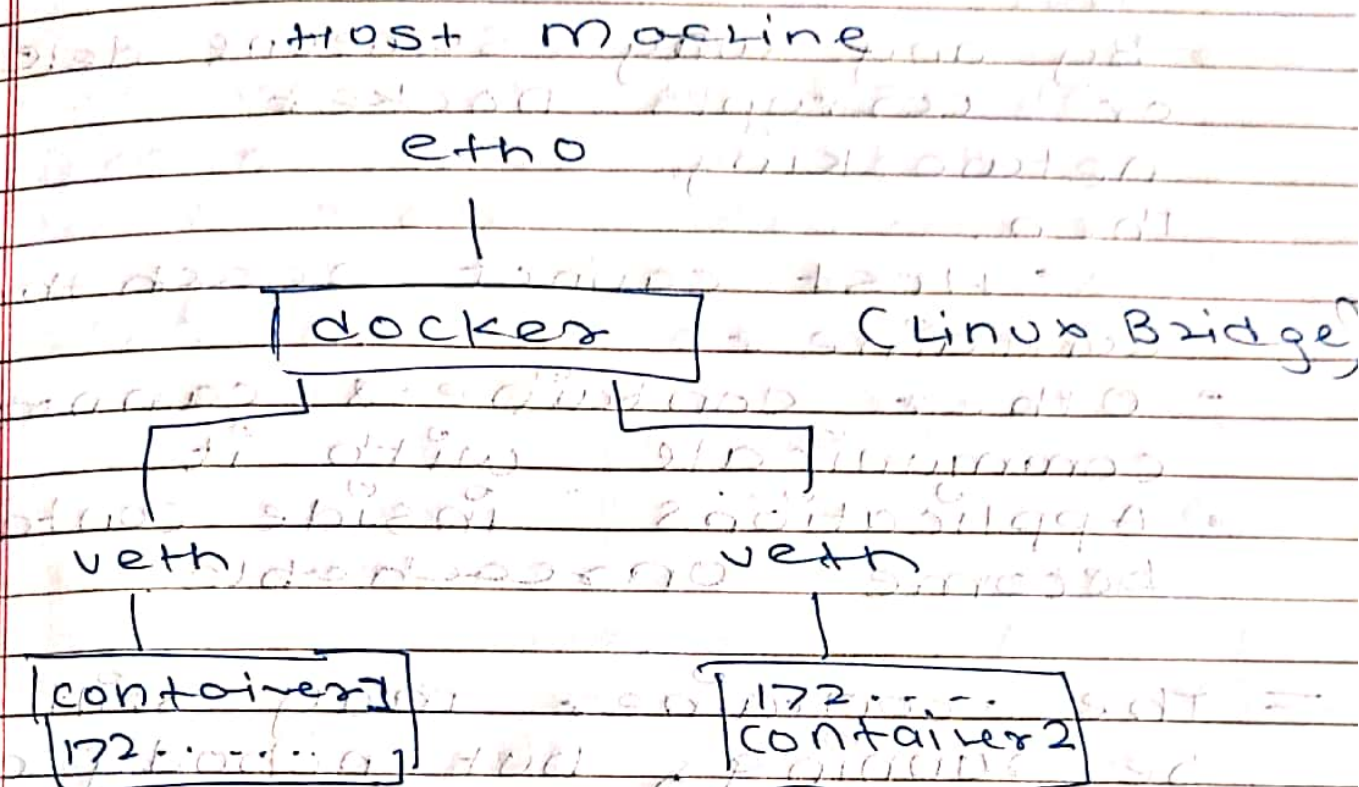
= Every container gets its own IP address.

= Host also has its own IP address.

- Since they are different network endpoints, Docker needs something that connects them.

↳ the connector is called the Docker Bridge.

Bridge Network Diagram



Each container gets

- Its own IP
- Its own virtual ethernet connection (veth)

Both connects to the Docker bridge (docker0)

Bridge Networking — It is called this because Docker uses a Linux bridge (docker0) to connect container network interfaces together & provide connectivity. IP's are assigned to containers through bridge network subnet.

— What if networking is deleted?

→ By any way someone delete or corrupts Docker networking.
Then.

- Host cannot reach the container
- Other containers cannot communicate with it.
- Applications inside container become unreachable.

= The container may still be running, but nobody can access the application.

~~Host Networking~~
Host Networking — Docker

Host Networking

Instead of creating an isolated network, the container directly uses the host's networking stack.

- There is — no bridge
- No separate container
- No veth pair

→ The application directly uses the host's network.

Problems with Bridge Networking

Example:- lets say we have
2 containers

- Login
- Finance

Both are connected to the
same default bridge network

So both are connected to Docker

C1 $\longleftrightarrow$ Docker0 $\longleftrightarrow$ C2

they can communicate
with each other.

Security concern — If login
gets compromised, since it is on
same bridge network, an
attacker may be able to
communicate with the
Finance container.

Solution - Custom Bridge Network

- Docker allows you to create your own bridge network instead of a default bridge, you can create separate logical networks.

Containers on diff custom bridge networks cannot communicate with each other by default.

while they can still communicate with the host as permitted by Docker networking.

Commands

`docker run --network`

`my-network nginx`

tells the Docker network container should join. Docker manages how that network connects to the host.

Overlay Networking

It is mainly useful when containers are running on multiple hosts.

This is commonly used with:

- Docker Swarm
- Kubernetes